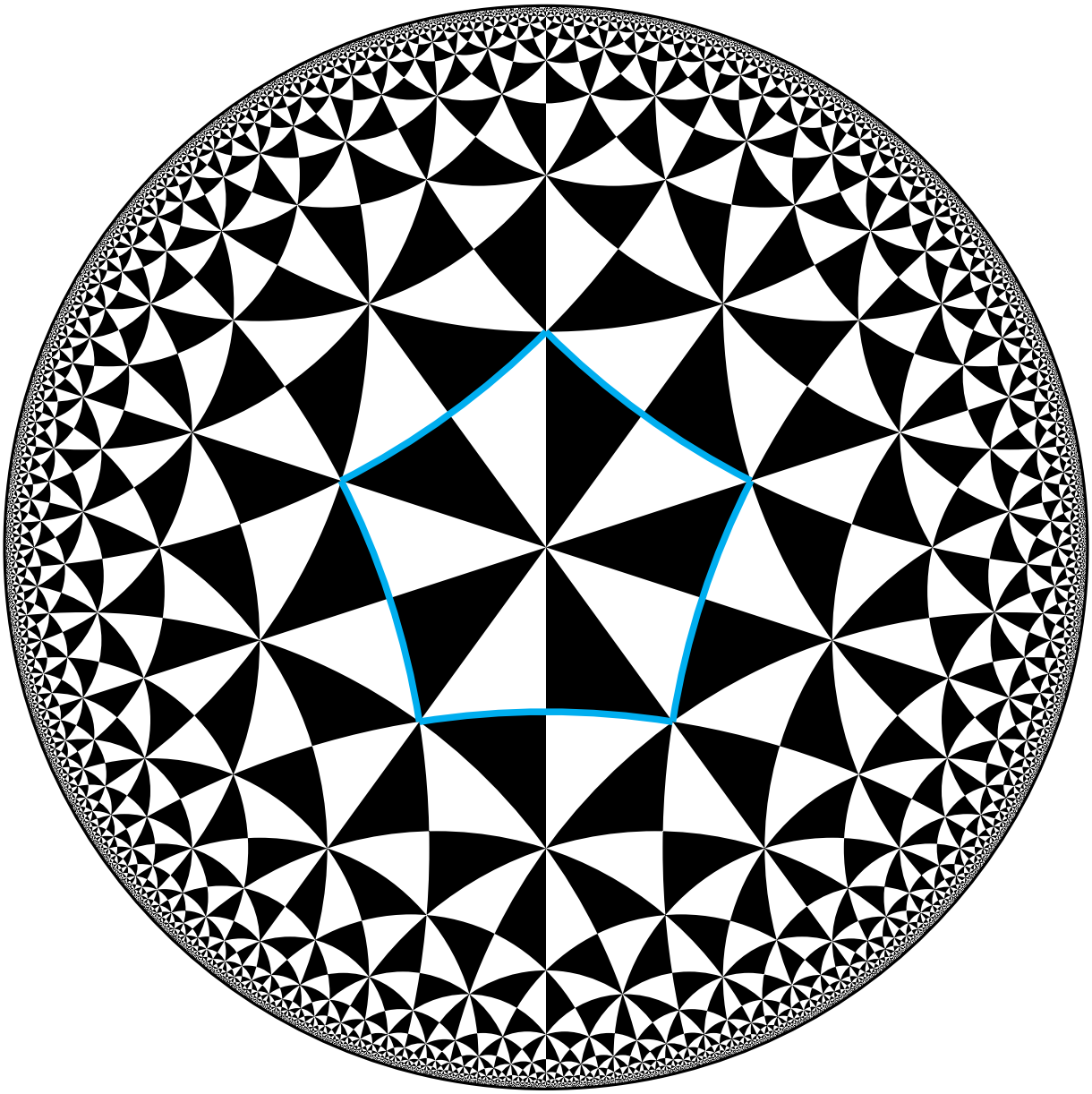
Package luahyperbolic

Damien Mégy

February 28, 2026



Contents

1 Introduction 2
1.1 What you can do with this package . . . 2
1.2 What this package cannot do for you . . . 2
1.3 How to load the package . . . 2
1.4 How to customize the package . . . 3
1.5 About the code and pictures shown in this manual . . . 3

2 Drawing elements 3
2.1 Points . . . 3
2.2 Geodesic segments, lines and rays . . . 4
2.3 Circles, semicircles, arcs . . . 7
2.4 Horocycles . . . 9

3 Defining points 9
3.1 Midpoint, interpolate, barycenter . . . 9
3.2 Triangle points (CAUTION) . . . 10
3.3 Intersections . . . 11
3.4 Reflections and projections . . . 12
3.5 (Elliptic) Rotations . . . 13
3.6 Orbits under automorphisms . . . 14

4 Tilings 15

5 Lua libraries 16
5.1 complex.lua . . . 16
5.1.1 Functions . . . 16
5.1.2 Constants (numbers and complex numbers) . . . 16
5.2 luahyperbolic-tikz.lua and hyperbolic-core . . . 17
5.2.1 Functions . . . 17
5.2.2 Constants (numbers and complex numbers) . . . 17

1 Introduction

1.1 What you can do with this package

:

1. Draw hyperbolic geometry pictures, high precision (pdf output), tiny size on disk
2. ...that you can later convert to svg or raster to bitmap images, if you want.
3. Easy (?) syntax. At least, easier than with LaTeXmacros.
4. Easy to read, educational Lua source code if you want to learn hyperbolic geometry.
5. No need to install Python packages or create an account on some webapp.
6. Compile anywhere in a single pass with lualatex compiler, available everywhere. No (Leeloo Dallas) multipass.
7. Doesn't need to compile in "shell-escape" mode.

1.2 What this package cannot do for you

1. Intensive computations. Source code is not optimized to keep maximum readability. All computations involve automorphisms. For heavy computations, please use a ~~Python~~ C library. The above (embedded) image compiled in approx. 10 seconds on a 2020 intel macbook air : 1.7 seconds to compute the 3783 geodesics wit Lua, and 8 seconds to draw them with TikZ. This documentation compiles in 10s including all the tilings in last section.
2. Coffee

1.3 How to load the package

Local install

Simply put the `luahyperbolic.sty` file in your working directory and add the following line to your tex preamble :

```
\usepackage{luahyperbolic}
```

The file `luahyperbolic.sty` contains everything needed, including the necessary library for complex numbers. It can be sent to colleagues by email etc.

It will declare two lua globals : `complex` and `hyper`, and define a `hyperbolicdisk` environment for convenience.

If the sty file is not in the same directory as your tex file, you can write `\usepackage{../luahyperbolic}`

CTAN

: not yet available

1.4 How to customize the package

The `luahyperbolic.sty` is generated from three lua modules : `complex.lua`, `luahyperbolic-core.lua` and `luahyperbolic-tikz.lua`. These files are in the `lua/` subdirectory of the package repo.

To build the `luahyperbolic.sty` from lua source files, execute `lua build.lua` in the root directory of the package. This will backup the old sty file to the `backup/` directory, then buid and replace the existing sty file.

1.5 About the code and pictures shown in this manual

In this documentation, we use the shortcut environment `\begin{hyperbolicdisk} ... \end{hyperbolicdisk}` which is defined in the package as follows:

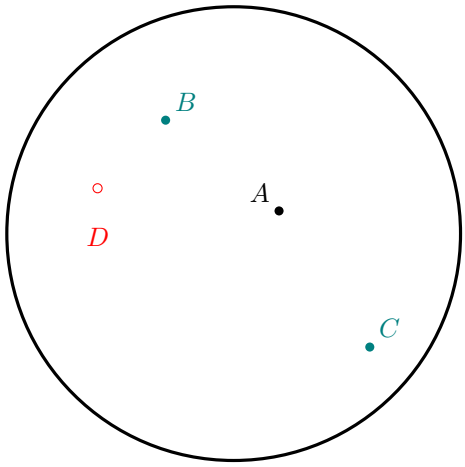
```
\newenvironment{hyperbolicdisk}[1][scale=3]{%
  \begin{tikzpicture}[#1]
    \draw[very thick] (0,0) circle (1);
    \clip (0,0) circle (1);
}{%
  \end{tikzpicture}
}
```

2 Drawing elements

2.1 Points

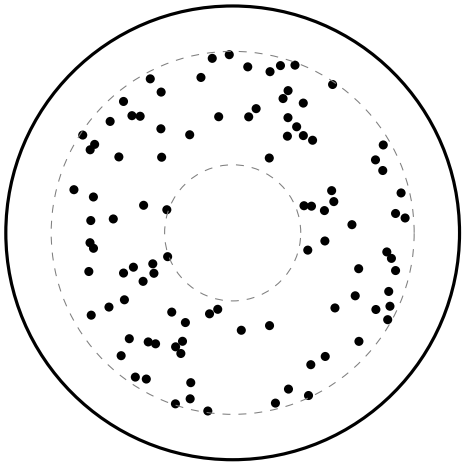
Draw and label points

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(0.2,0.1)
local B = complex(-0.3,0.5)
local C = complex(0.6,-0.5)
local D = complex(-0.6,0.2)
hyper.drawPoint(A)
hyper.drawPoints(B, C, "teal")
hyper.drawPoint(D, "white, draw=red")
hyper.labelPoint(A,"$A$") -- default : above
    ↪ left
hyper.labelPoints(B,"$B$", C, "$C$", "above
    ↪ right, teal")
hyper.labelPoint(D,"$D$","below=.4cm, red")
-- hyper.drawPoint(complex(2,2)) -- will error
\end{luacode*}
\end{hyperbolicdisk}
```



Random points with uniform density

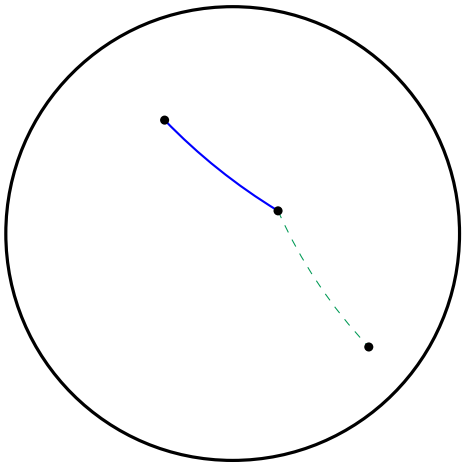
```
\begin{hyperbolicdisk}
\begin{luacode*}
for k=1,100 do
  local M = hyper.randomPoint(0.3,0.8)
  hyper.drawPoint(M)
end
\end{luacode*}
\draw[gray, dashed] (0,0) circle(0.3) circle
    ↪ (0.8);
\end{hyperbolicdisk}
```



2.2 Geodesic segments, lines and rays

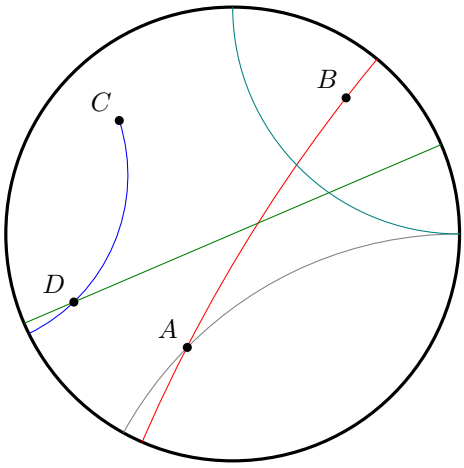
Geodesic segments

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(0.2,0.1)
local B = complex(-0.3,0.5)
local C = complex(0.6,-0.5)
hyper.drawSegment(A, B, "thick,blue")
hyper.drawSegment(A, C, "dashed,ForestGreen")
hyper.drawPoints(A,B,C)
\end{luacode*}
\end{hyperbolicdisk}
```



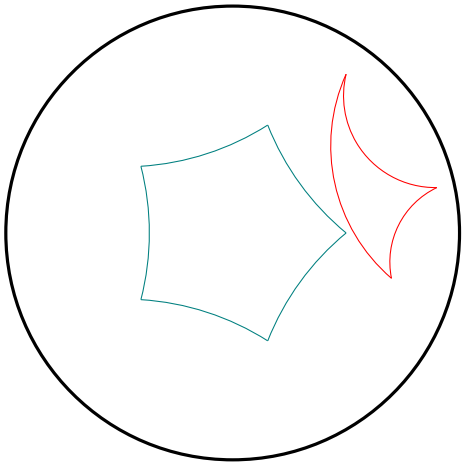
Lines and rays (through two points)

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(-0.2,-0.5)
local B = complex(0.5,0.6)
local C = complex(-0.5,0.5)
local D = complex(-0.7,-0.3)
hyper.drawLine(A, B, "red")
hyper.drawRay(C,D, "blue")
hyper.drawLine(D,-D, "green!50!black")
hyper.drawLine(A,complex(1,0), "gray")
hyper.drawLine(complex(1,0), complex(0,1), "
    ↪ teal")
hyper.drawPoints(A, B, C, D)
hyper.labelPoints(A, "$A$", B, "$B$", C, "$C$",
    ↪  D, "$D$")
\end{luacode*}
\end{hyperbolicdisk}
```



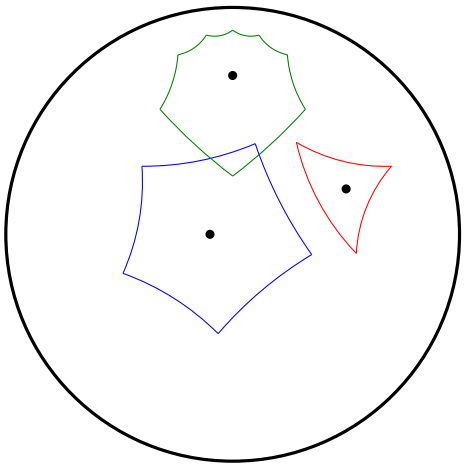
Triangles and polygons

```
\begin{hyperbolicdisk}
\begin{luacode*}
hyper.drawTriangle(
  complex(.9,.2),
  complex(.5,.7),
  complex(.7,-.2), "red")
local points={}
for k=1,5 do
  table.insert(points, complex.polar(.5,k*2*
      ↪ math.pi/5))
end
hyper.drawPolygonFromTable(points,"teal")
\end{luacode*}
\end{hyperbolicdisk}
```



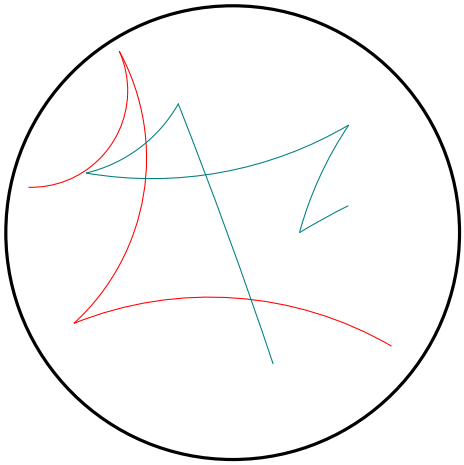
Regular polygons

```
\begin{hyperbolicdisk}
\begin{luacode*}
local O1 = complex(.5,.2)
local A1 = complex(.7,.3)
local O2 = complex(-0.1,0)
local A2 = complex(-.4,.3)
local O3 = complex(0,.7)
local A3 = complex(0,.9)
hyper.drawRegularPolygon(O1, A1, 3, "red")
hyper.drawRegularPolygon(O2, A2, 5, "blue")
hyper.drawRegularPolygon(O3, A3, 8, "green!50!
    ↪ black")
hyper.drawPoints(O1,O2,O3)
\end{luacode*}
\end{hyperbolicdisk}
```



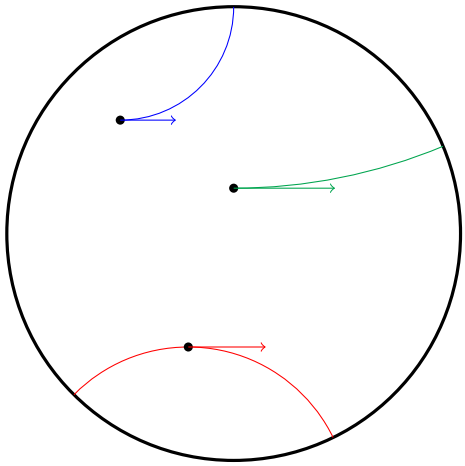
Polygonal lines

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(-.9,.2)
local B = complex(-.5,.8)
local C = complex(-.7,-.4)
local D = complex(.7,-.5)
hyper.drawPolyline(A, B, C, D, "red")
local points={}
for k=1,6 do
  table.insert(points, hyper.randomPoint(0,.7))
end
hyper.drawPolylineFromTable(points,"teal")
\end{luacode*}
\end{hyperbolicdisk}
```



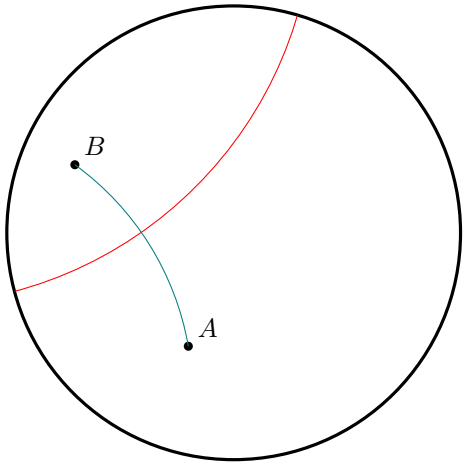
Lines and rays (point and tangent vector)

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(-0.2,-0.5)
local B = complex(-0.5,0.5)
local C = complex(0,0.2)
local v =  complex(1,0)
hyper.drawLineFromVector(A, v, "red")
hyper.drawRayFromVector(B, v, "blue")
hyper.drawRayFromVector(C, v, "Green")
hyper.drawPoints(A, B,C)
hyper.drawTangentVector(A,v,"red")
hyper.drawTangentVector(B,v,"blue")
hyper.drawTangentVector(C,v,"Green")
\end{luacode*}
\end{hyperbolicdisk}
```



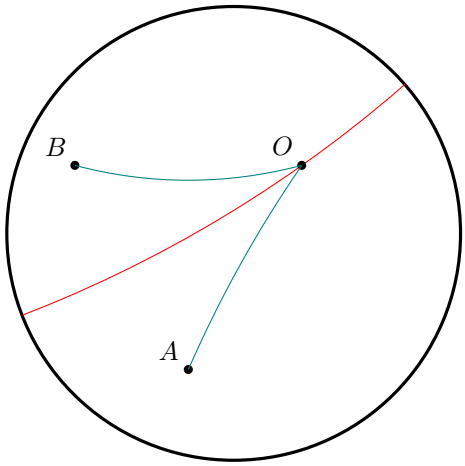
Perpendicular bisector

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(-0.2,-0.5)
local B = complex(-0.7,0.3)
hyper.drawPerpendicularBisector(A,B,"red")
hyper.drawPoints(A,B)
hyper.drawSegment(A, B, "teal")
hyper.labelPoints(A,"$A$", B, "$B$", "above
    ↪ right")
\end{luacode*}
\end{hyperbolicdisk}
```



Angle bisector

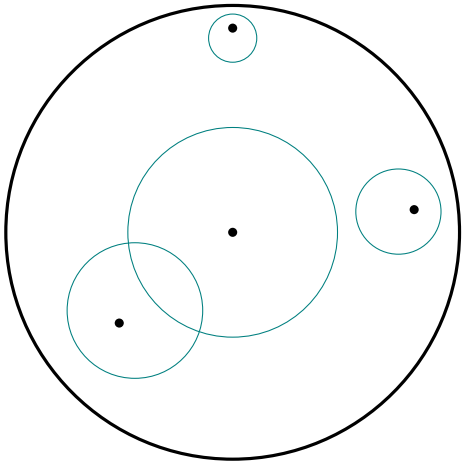
```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(-0.2,-0.6)
local O = complex(0.3,0.3)
local B = complex(-0.7,0.3)
hyper.drawAngleBisector(A,O,B,"red")
hyper.drawPoints(A,O,B)
hyper.drawPolyline(A,O,B, "teal")
hyper.labelPoints(A, "$A$", O, "$O$", B, "$B$")
\end{luacode*}
\end{hyperbolicdisk}
```



2.3 Circles, semicircles, arcs

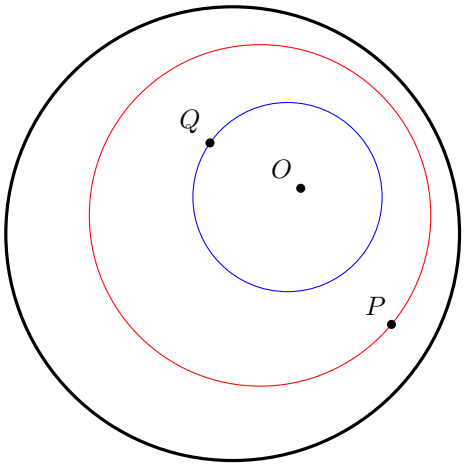
Center and radius

```
\begin{hyperbolicdisk}
\begin{luacode*}
local points = {
  complex(0,0),
  complex(-0.5,-0.4),
  complex(0.8,0.1),
  complex(0,0.9)
}
for _,point in ipairs(points) do
  hyper.drawCircle(point,1,"teal")
  hyper.drawPoint(point)
end
-- NOTE : all circles have same radius=1 !
\end{luacode*}
\end{hyperbolicdisk}
```



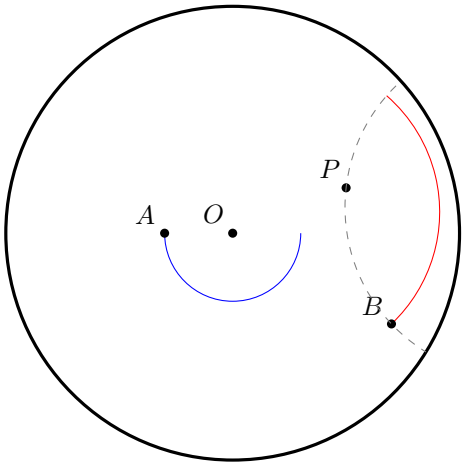
Circle through point

```
\begin{hyperbolicdisk}
\begin{luacode*}
local O = complex(0.3,0.2)
local P = complex(0.7,-0.4)
local Q = complex(-0.1,0.4)
hyper.drawCircleThrough(O, P, "red")
-- or by hand :
hyper.drawCircle(O, hyper.distance(O,Q), "blue"
    ↪ )
hyper.drawPoints(O, P, Q)
hyper.labelPoints(O, "$O$", P, "$P$", Q, "$Q$")
\end{luacode*}
\end{hyperbolicdisk}
```



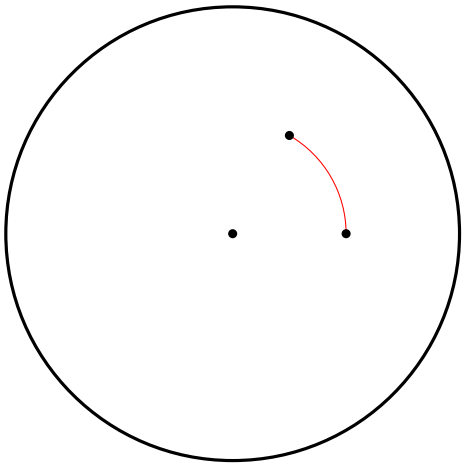
Semicircles

```
\begin{hyperbolicdisk}
\begin{luacode*}
local O = complex(0,0)
local A = complex(-0.3,0)
local P = complex(0.5,0.2)
local B = complex(0.7,-0.4)
hyper.drawSemicircle(O, A, "blue")
hyper.drawSemicircle(P, B, "red")
hyper.drawPoints(O, A, P, B)
hyper.labelPoints(O, "$O$", A, "$A$", P, "$P$",
    ↪  B, "$B$")
hyper.drawLine(P,B, "gray, dashed")
\end{luacode*}
\end{hyperbolicdisk}
```



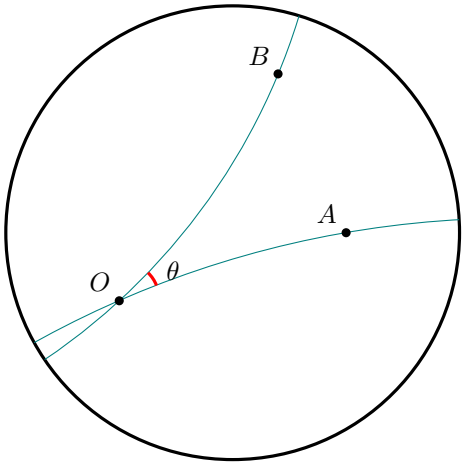
Arc (center and endpoints)

```
\begin{hyperbolicdisk}
\begin{luacode*}
local O = complex(0,0)
local A = complex(0.5,0)
local B = complex.polar(0.5,math.pi/3)
hyper.drawArc(O,A,B, "red")
hyper.drawPoints(O,A,B)
\end{luacode*}
\end{hyperbolicdisk}
```



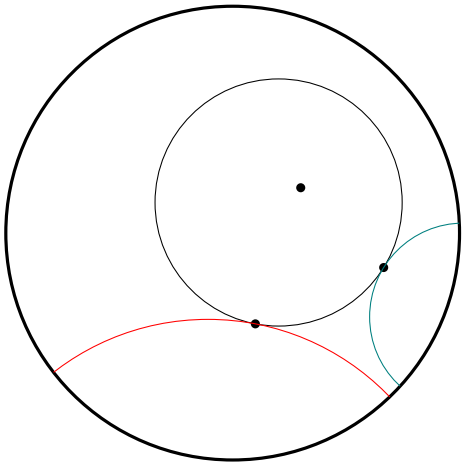
Draw angle $\widehat{AOB}$

```
\begin{hyperbolicdisk}
\begin{luacode*}
local O = complex(-.5,-.3)
local A = complex(0.5,0)
local B = complex(0.2,.7)
hyper.drawLines(O, A, O, B, "teal")
hyper.drawAngle(A,O,B, "red, very thick")
hyper.drawPoints(A,O,B)
hyper.labelPoints(A, "$A$", O, "$O$", B, "$B$")
-- poor man's labelAngle:
hyper.labelPoint(O,"$\\theta$", "above=.4cm,
    ↪ right=.5cm")
\end{luacode*}
\end{hyperbolicdisk}
```



Tangent at point

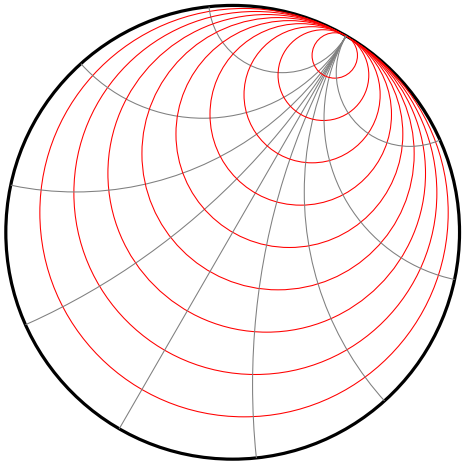
```
\begin{hyperbolicdisk}
\begin{luacode*}
local O = complex(0.3,0.2)
local P = complex(0.1,-0.4)
local Q = hyper.rotation(O,math.pi/3)(P)
hyper.drawCircleThrough(O, P)
hyper.drawPoints(O, P, Q)
hyper.drawTangentAt(O,P,"red")
hyper.drawTangentAt(O,Q,"teal")
\end{luacode*}
\end{hyperbolicdisk}
```



2.4 Horocycles

Horocycles

```
\begin{hyperbolicdisk}
\begin{luacode*}
local theta=math.pi/3
local idealPoint = complex.exp_i(theta)
local N = 10
for k=1,N-1 do
  local point = idealPoint * (2*k/N-1)
  hyper.drawHorocycle(idealPoint,point,"red")
  -- and an orthogonal geodesic :
  local endPoint = complex.exp_i(theta+2*k*math
      ↪ .pi/N)
  hyper.drawLine(idealPoint,endPoint,"gray")
end
\end{luacode*}
\end{hyperbolicdisk}
```

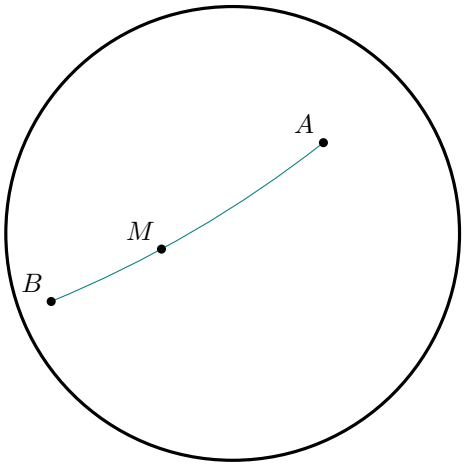


3 Defining points

3.1 Midpoint, interpolate, barycenter

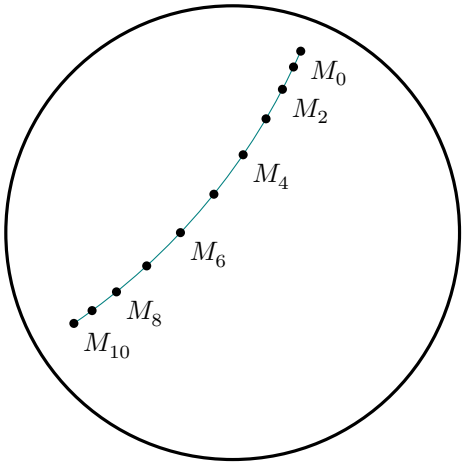
Midpoint

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(0.4,0.4)
local B = complex(-0.8,-0.3)
local M = hyper.midpoint(A,B)
hyper.drawSegment(A,B, "teal")
hyper.drawPoints(A,B,M)
hyper.labelPoints(A, "$A$", B, "$B$", M, "$M$")
\end{luacode*}
\end{hyperbolicdisk}
```



Interpolate and barycenter

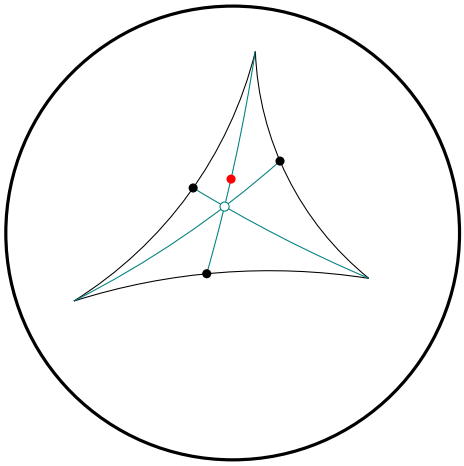
```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(0.3,0.8)
local B = complex(-0.7,-0.4)
hyper.drawSegment(A,B, "teal")
for k=0,10 do
  local M = hyper.interpolate(A, B, k/10)
  hyper.drawPoint(M)
  if k % 2 == 0 then
    hyper.labelPoint(M, "$M_{" .. k .. "}$", "
        ↪ below right")
  end
end
\end{luacode*}
\end{hyperbolicdisk}
```



3.2 Triangle points (CAUTION)

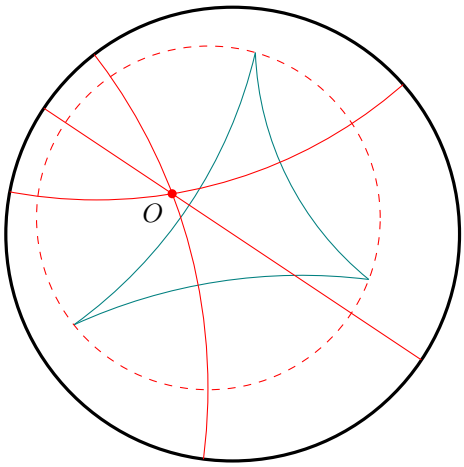
Centroid (intersection of medians)

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(0.1,0.8)
local B = complex(-0.7,-0.3)
local C = complex(0.6,-0.2)
local AA = hyper.midpoint(B,C)
local BB = hyper.midpoint(C,A)
local CC = hyper.midpoint(A,B)
local G = hyper.triangleCentroid(A,B,C)
hyper.drawTriangle(A,B,C)
hyper.drawSegments(A, AA, B, BB, C, CC, "teal")
hyper.drawPoints(AA,BB,CC)
hyper.drawPoint(G,"white, draw=teal")
-- CAUTION :
local wrong = hyper.interpolate(A,AA,2/3)
hyper.drawPoint(wrong, "red")
\end{luacode*}
\end{hyperbolicdisk}
```



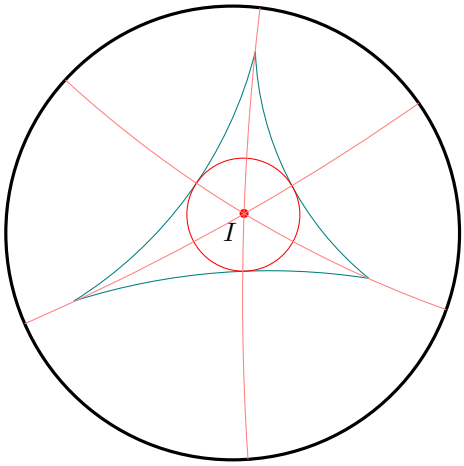
Circumcenter and circumcircle (CAUTION : does not always exist)

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(0.1,0.8)
local B = complex(-0.7,-0.4)
local C = complex(0.6,-0.2)
hyper.drawTriangle(A,B,C,"teal")
hyper.drawPerpendicularBisector(A,B, "red")
hyper.drawPerpendicularBisector(B,C, "red")
hyper.drawPerpendicularBisector(C,A, "red")
local O = hyper.circumcenter(A,B,C)
-- Does not always exist !!
if O ~= nil then
  hyper.drawPoint(O,"red")
  local r = hyper.distance(O,A)
  hyper.drawCircle(O,r,"red, dashed")
  hyper.labelPoint(O,"$O$", "below left")
end
\end{luacode*}
\end{hyperbolicdisk}
```



Incenter and incircle

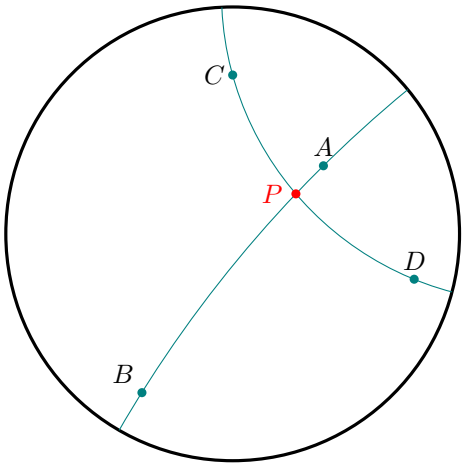
```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(0.1,0.8)
local B = complex(-0.7,-0.3)
local C = complex(0.6,-0.2)
local I = hyper.incenter(A,B,C)
hyper.drawTriangle(A,B,C,"teal")
hyper.drawPoint(I,"red")
hyper.labelPoint(I,"$I$", "below left")
hyper.drawIncircle(A,B,C,"red")
-- show intersection :
hyper.drawAngleBisector(A,B, C, "red!50")
hyper.drawAngleBisector(B,C,A, "red!50")
hyper.drawAngleBisector(C,A,B, "red!50")

\end{luacode*}
\end{hyperbolicdisk}
```



3.3 Intersections

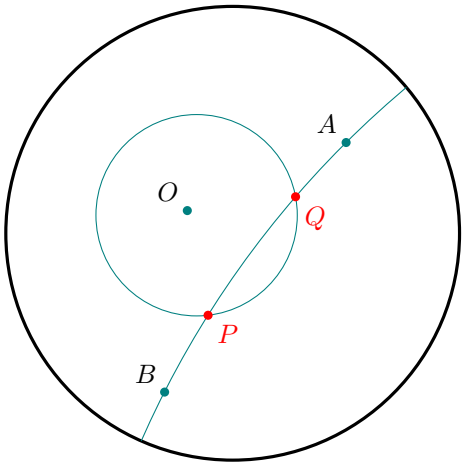
Line-Line (LL) intersection

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(0.4,0.3)
local B = complex(-0.4,-0.7)
local C = complex(0,0.7)
local D = complex(0.8,-0.2)
local P = hyper.interLL(A, B, C, D)
hyper.drawLines(A, B, C, D, "teal")
hyper.drawPoints(A, B, C, D,"teal")
hyper.drawPoint(P,"red")
hyper.labelPoint(A, "$A$", "above")
hyper.labelPoint(B, "$B$")
hyper.labelPoint(C, "$C$", "left")
hyper.labelPoint(D, "$D$", "above")
hyper.labelPoint(P, "$P$", "left=.1cm, red")
\end{luacode*}
\end{hyperbolicdisk}
```



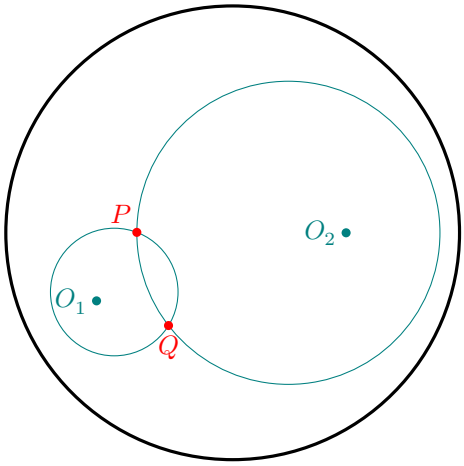
Line-circle (LC) intersection

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(0.5,0.4)
local B = complex(-0.3,-0.7)
local O = complex(-0.2,0.1)
local radius = 1
local P, Q = hyper.interLC(A, B, O, radius)
hyper.drawLine(A, B, "teal")
hyper.drawCircle(O,radius, "teal")
hyper.drawPoints(A, B, O, "teal")
hyper.drawPoints(P, Q, "red")
hyper.labelPoints(A, "$A$", B, "$B$", O, "$O$")
hyper.labelPoints(P, "$P$", Q, "$Q$", "red,
    ↪ below right")
\end{luacode*}
\end{hyperbolicdisk}
```



Circle-circle (CC) intersection

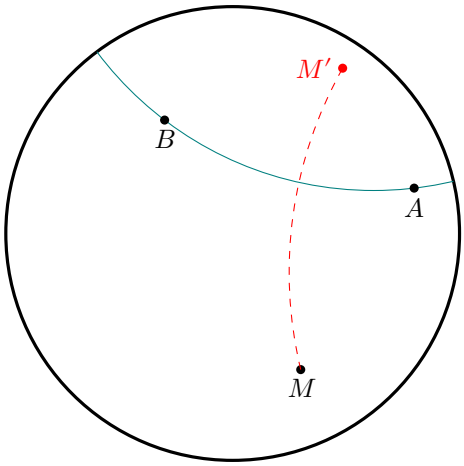
```
\begin{hyperbolicdisk}
\begin{luacode*}
local O1 = complex(-0.6,-.3)
local O2 = complex(0.5,0)
local r1, r2 = 1, 2
local P, Q = hyper.interCC(O1, r1, O2, r2)
hyper.drawCircle(O1,r1, "teal")
hyper.drawCircle(O2,r2, "teal")
hyper.drawPoints(P, Q, "red")
hyper.drawPoints(O1, O2, "teal")
hyper.labelPoints(O1, "$O_1$", O2, "$O_2$", "
    ↪ left, teal")
hyper.labelPoints(P, "$P$", "above left, red")
hyper.labelPoint(Q, "$Q$", "red, below")
\end{luacode*}
\end{hyperbolicdisk}
```



3.4 Reflections and projections

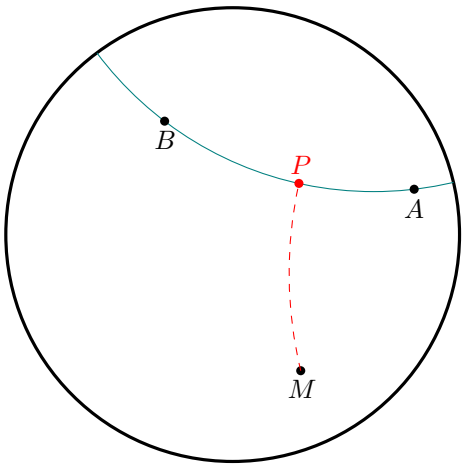
Reflection across a geodesic line

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(0.8,0.2)
local B = complex(-0.3,0.5)
local M = complex(0.3,-0.6)
local Mp = (hyper.reflection(A,B))(M)
hyper.drawLine(A, B, "teal")
hyper.drawPoints(A, B, M)
hyper.drawPoint(Mp,"red")
hyper.drawSegment(M,Mp,"dashed,red")
hyper.labelPoint(Mp,"$M'$", "left, red")
hyper.labelPoints(A, "$A$", B, "$B$", M, "$M$",
    ↪  "below")
\end{luacode*}
\end{hyperbolicdisk}
```



Projection onto a geodesic line

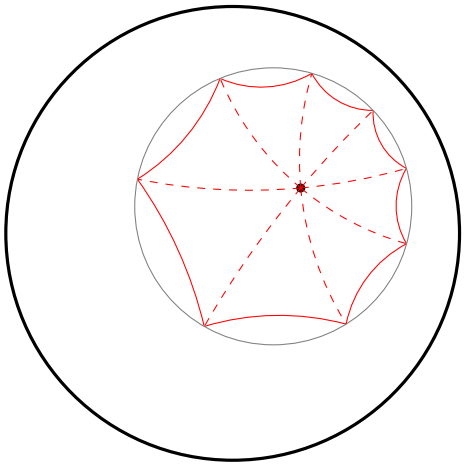
```
\begin{hyperbolicdisk}
\begin{luacode*}
local A = complex(0.8,0.2)
local B = complex(-0.3,0.5)
local M = complex(0.3,-0.6)
local P = (hyper.projection(A,B))(M)
hyper.drawLine(A, B, "teal")
hyper.drawPoints(A, B, M)
hyper.drawPoint(P,"red")
hyper.drawSegment(M,P,"dashed,red")
hyper.labelPoint(P,"$P$", "above, red")
hyper.labelPoints(A, "$A$", B, "$B$", M, "$M$",
    ↪  "below")
\end{luacode*}
\end{hyperbolicdisk}
```



3.5 (Elliptic) Rotations

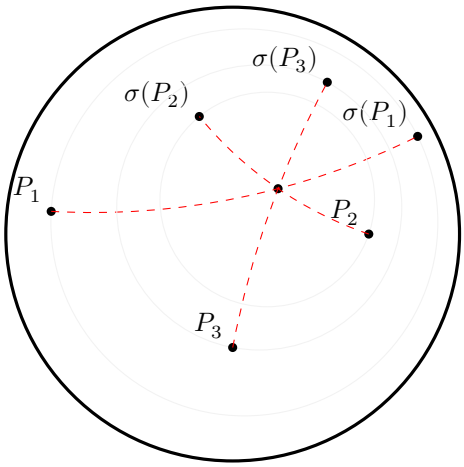
(Elliptic) rotation around a point (regular octogon)

```
\begin{hyperbolicdisk}
\begin{luacode*}
local O = complex(0.3,0.2)
local M = complex(0.5,-0.4)
hyper.drawPoints(O)
for k=1,8 do
  local P = (hyper.rotation(O,2*math.pi/8))(M)
  hyper.drawSegment(M, P, "red")
  hyper.drawSegment(O,M, "dashed,red")
  M = P
end
hyper.drawCircle(O,hyper.distance(O,M),"thin,
    ↪ gray")
\end{luacode*}
\end{hyperbolicdisk}
```



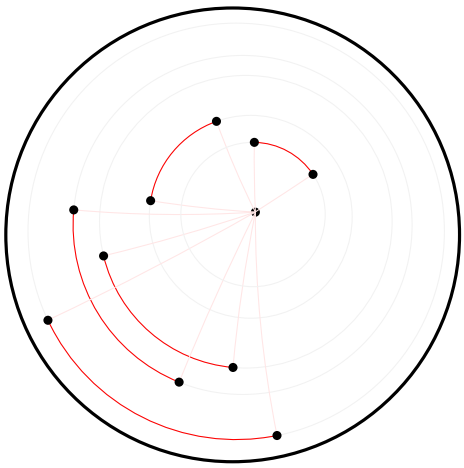
Symmetry around a point (rotation by 180°)

```
\begin{hyperbolicdisk}
\begin{luacode*}
local O = complex(0.2,0.2)
local s = hyper.symmetry(O)
hyper.drawPoints(O)
local points={complex(-.8,.1), complex(.6,0),
    ↪ complex(0,-.5)}
for k, P in ipairs(points) do
  hyper.drawCircleThrough(O, P, "gray!10")
  local sP = s(P)
  hyper.drawPoints(P,sP)
  hyper.drawSegment(P,sP, "red, dashed")
  hyper.labelPoint(P, "$P_{".. k .."}$")
  hyper.labelPoint(sP, "$\\sigma(P_{".. k .."})
      ↪ $")
end
\end{luacode*}
\end{hyperbolicdisk}
```



Rotation by 60°

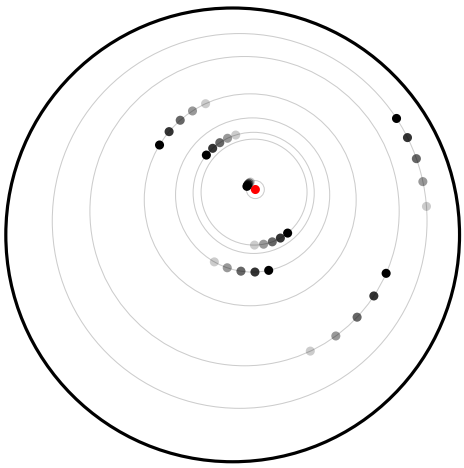
```
\begin{hyperbolicdisk}
\begin{luacode*}
local O = complex(0.1,0.1)
local r = hyper.rotation(O,math.pi/3)
hyper.drawPoints(O)
for k=1,5 do
  local P = hyper.randomPoint(0.3,0.9)
  local rP = r(P)
  hyper.drawCircleThrough(O,P, "gray!10")
  hyper.drawArc(O,P,rP,"red")
  hyper.drawPolyline(P,O,rP, "red!10")
  hyper.drawPoints(P,rP)
end
\end{luacode*}
\end{hyperbolicdisk}
```



3.6 Orbits under automorphisms

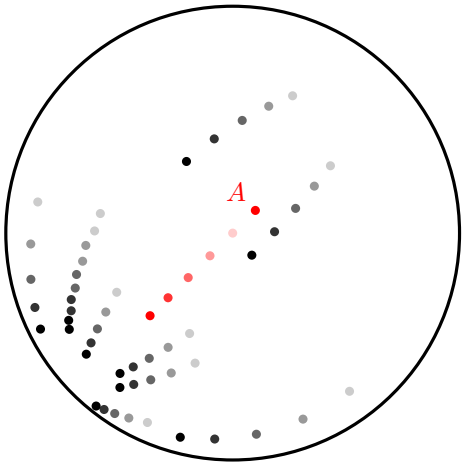
Orbit under elliptic automorphism (rotation)

```
\begin{hyperbolicdisk}
\begin{luacode*}
local nbPoints = 7
local nbIterations = 5
local O = complex(0.1,0.2)
local rot = hyper.rotation(O,math.pi/20) --
    ↪ angle in radians
hyper.drawPoint(O,"red")
for k=1,nbPoints do
  local P = hyper.randomPoint(0,.9)
  hyper.drawCircleThrough(O,P,"draw opacity=0.2
      ↪ ")
  hyper.drawPointOrbit(P, rot, nbIterations)
end
\end{luacode*}
\end{hyperbolicdisk}
```



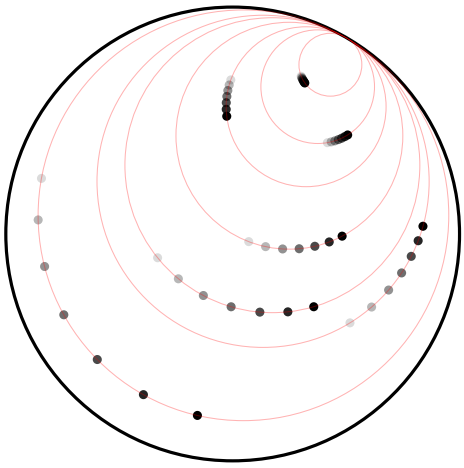
Orbit under hyperbolic automorphism

```
\begin{hyperbolicdisk}
\begin{luacode*}
local nbPoints = 10
local nbIterations = 5
local A = complex(0.1,0.1)
local phi = hyper.automorphism(A,0)
for k=1,nbPoints do
  local P = hyper.randomPoint(0,.9)
  hyper.drawPointOrbit(P, phi, nbIterations)
end
hyper.drawPoint(A,"red")
hyper.labelPoint(A,"$A$", "above left, red")
hyper.drawPointOrbit(A, phi, nbIterations, "red
    ↪ ")
\end{luacode*}
\end{hyperbolicdisk}
```



Orbit under parabolic automorphism

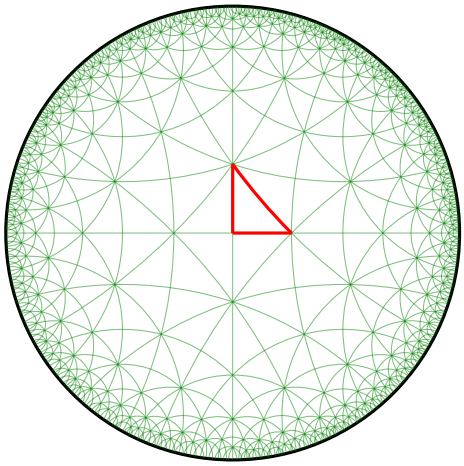
```
\begin{hyperbolicdisk}
\begin{luacode*}
local nbIterations = 7
limit = complex.exp_i(math.pi/3) -- on circle
local phi = hyper.parabolic(limit,math.pi/20)
local points={
  complex(0,0), complex(0,.7),
  complex(.4,.4), complex(.3,.7),
  complex(-.8,.4), complex(-.4,0),
  complex(.4,-.45)}
for _,P in ipairs(points) do
  hyper.drawPointOrbit(P, phi, nbIterations)
  hyper.drawHorocycle(limit, P, "red, draw
      ↪ opacity=.3")
end
\end{luacode*}
\end{hyperbolicdisk}
```



4 Tilings

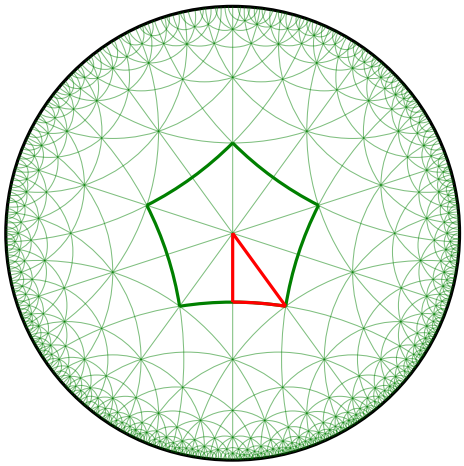
Triangle tiling

```
\begin{hyperbolicdisk}
\begin{luacode*}
hyper.SCALE=1e5
local A, B, C = hyper.fundamentalRightTriangle
    ↪ (4,5)
local geodesics = {
{hyper.endpoints(A,B)},
  {hyper.endpoints(B,C)},
  {hyper.endpoints(C,A)}}
geodesics = hyper.propagate_geodesics(geodesics
    ↪ , TILING_DEPTH)
hyper.drawLinesFromTable(geodesics, "green!50!
    ↪ black, draw opacity=.5")
hyper.drawTriangle(A,B,C,"red, very thick")
\end{luacode*}
\end{hyperbolicdisk}
```



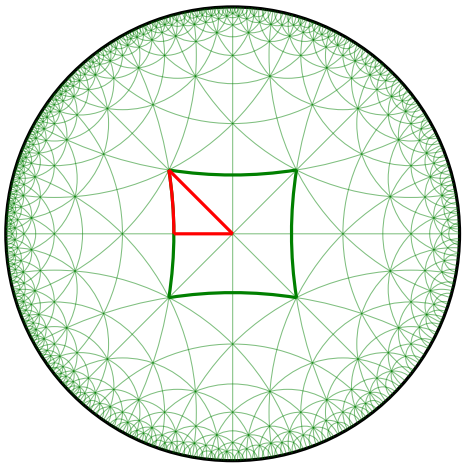
Same tiling but centered on the pentagon

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A, B, C = hyper.fundamentalRightTriangle
    ↪ (4,5)
local phi = hyper.automorphism(C,0)
A, B, C = phi(A), phi(B), phi(C)
local geodesics = {
  {hyper.endpoints(A,B)},
  {hyper.endpoints(B,C)},
  {hyper.endpoints(C,A)}}
geodesics = hyper.propagate_geodesics(geodesics
    ↪ , TILING_DEPTH)
hyper.drawLinesFromTable(geodesics, "green!50!
    ↪ black, draw opacity=.5")
hyper.drawRegularPolygon(complex(0,0),B,5, "
    ↪ very thick, green!50!black")
hyper.drawTriangle(A,B,C,"red, very thick")
\end{luacode*}
\end{hyperbolicdisk}
```



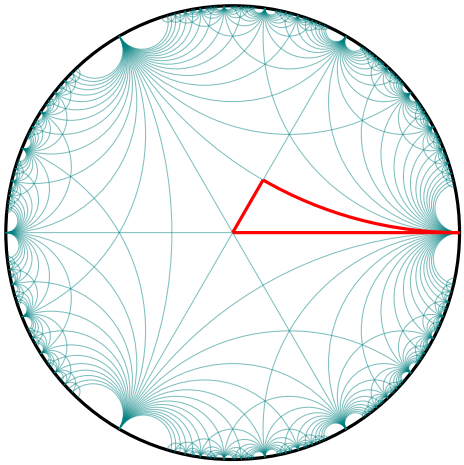
Same tiling but centered on the square

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A, B, C = hyper.fundamentalRightTriangle
    ↪ (4,5)
local phi = hyper.automorphism(B,0)
A, B, C = phi(A), phi(B), phi(C)
local geodesics = {
  {hyper.endpoints(A,B)},
  {hyper.endpoints(B,C)},
  {hyper.endpoints(C,A)}
}
geodesics = hyper.propagate_geodesics(geodesics
    ↪ , TILING_DEPTH)
hyper.drawLinesFromTable(geodesics, "green!50!
    ↪ black, draw opacity=.5")
hyper.drawRegularPolygon(complex(0,0),C,4, "
    ↪ very thick, green!50!black")
hyper.drawTriangle(A,B,C,"red, very thick")
\end{luacode*}
\end{hyperbolicdisk}
```



Ideal triangle tiling

```
\begin{hyperbolicdisk}
\begin{luacode*}
local A, B, C = hyper.fundamentalIdealTriangle
    ↪ (3,2)
local geodesics = {
  {hyper.endpoints(A,B)},
  {hyper.endpoints(B,C)},
  {hyper.endpoints(C,A)}
}
geodesics = hyper.propagate_geodesics(geodesics
    ↪ , TILING_DEPTH)
hyper.drawLinesFromTable(geodesics, "teal, draw
    ↪  opacity=.5")
hyper.drawTriangle(A,B,C,"red, very thick")
\end{luacode*}
\end{hyperbolicdisk}
```



5 Lua libraries

5.1 complex.lua

5.1.1 Functions

__add	__unm	det	isClose	isReal	rotate
__div	abs	distinct	isClosePair	isUnit	rotate90
__eq	abs2	dot	isColinear	log	scal
__mul	arg	exp	isImag	new	toPolar
__sub	clone	exp_i	isNear	nonzero	unit
__tostring	conj	invert	isNot	polar	

5.1.2 Constants (numbers and complex numbers)

EPS, EPS_INV, I, J, ONE, ZERO, __index,

5.2 luahyperbolic-tikz.lua and hyperbolic-core

5.2.1 Functions

acosh
angle
asinh
assert
assert_in_disk
atanh
automorphism
automorphism_from_pairs
automorphism_sending
barycenter2
circle_to_euclidean
circumcenter
cosh
distance
distance_to_geodesic
distance_to_origin
drawAngle
drawAngleBisector
drawArc
drawCircle
drawCircleThrough
drawHorocycle
drawIncircle
drawLabeledPoint
drawLine
drawLineFromVector
drawLines
drawLinesFromTable
drawPerpendicularBisector
drawPoint
drawPointOrbit
drawPoints
drawPolySeg
drawPolySegFromTable
drawPolygon
drawPolygonFromTable
drawPolyline
drawPolylineFromTable
drawRay
drawRayFromPoints
drawRayFromVector
drawRegularPolygon
drawSegment
drawSegments
drawSemicircle
drawTangentAt
drawTangentVector
drawTriangle
endpoints
endpoints_angle_bisector
endpoints_perpendicular_bisector
ensure_order
error
exp_map
exp_map_at_origin
fundamentalIdealTriangle
fundamentalRightTriangle
geodesic_data
in_closed_disk
in_closed_half_plane
in_disk
in_half_plane
incenter
interCC
interLC
interLL
interpolate
is_on_geodesic
labelPoint
labelPoints
metric_factor
midpoint
midpoint_to_origin
mobius_transformation
on_circle
parabolic
parabolic_fix1
projection
propagate_geodesics
radial_half
radial_scale
randomPoint
reflection
rotation
rotation_from_pair
same_geodesics
shape_closed_line
shape_euclidean_segment
shape_segment
sinh
symmetry
symmetry_around_midpoint
tangent
tikzBegin
tikzBeginDisk
tikzEnd
tikz_define_nodes
triangleCentroid

5.2.2 Constants (numbers and complex numbers)

DRAW_EPS, DRAW_EPS_INV, DRAW_POINT_RADIUS, EPS, SCALE,